

When the Agent Decides: Five Design Failures in Autonomous Offensive Security Agents

Ricardo Martínez

Cloud Security & Applied AI

September 2026

Abstract

Autonomous AI agents for penetration testing promise to scale offensive work without scaling the human team. This piece documents five field observations, drawn from operating one such system against real targets over several months, on why the reliability of these agents depends less on the capability of the model orchestrating them and more on concrete engineering decisions: what the LLM is allowed to decide and what it isn't, how a finding gets verified before it is believed, and how silently the system degrades when something in its environment fails. Four figures grounded in real measurements are presented, closing with five design principles applicable to any agentic system operating against real attack surface.

1 The Mirage of Total Autonomy

The first instinct when building a pentesting agent is to let the language model decide which tool to use at each step. It is also the first source of a coverage problem that stays invisible until someone actually measures it.

In a system with roughly sixty available attack capabilities (injection, SSRF, deserialization, secret leakage, access control, JWT, GraphQL), auditing how many ran with guaranteed certainty on every run produced an uncomfortable number: fewer than twenty. Everything else only ran if the agent, in that moment, decided it was worth calling. One concrete test confirmed it: a domain reconnaissance technique showed up in one run and vanished in the next against the exact same target, with nothing in the environment having changed.

The engineering answer was not to tune the prompt so the agent would “remember” to use more tools (Figure 1). It was to pull high-value, low-false-positive-risk capabilities out of the LLM’s discretionary territory entirely and turn them into a deterministic sweep that always runs, in parallel, before the agent even enters the picture. The model is left for what genuinely requires judgment: chaining findings together, prioritizing ambiguous surface area, writing up context.

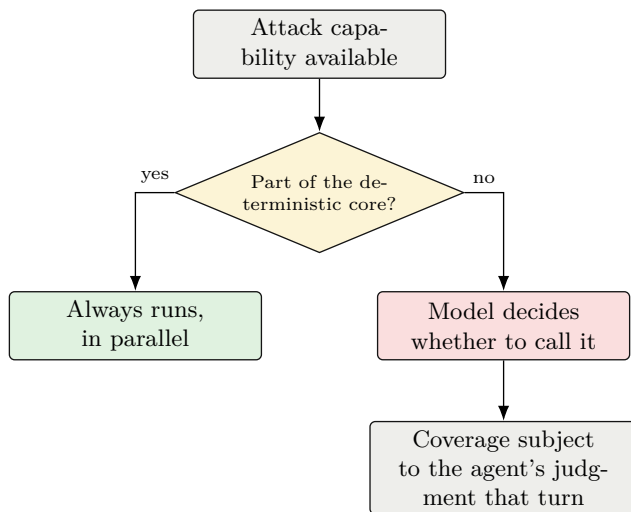


Figure 1: Coverage arbitration. Of ~60 available attack capabilities, fewer than 20 ran with guaranteed certainty on every run; the rest were subject to the agent’s judgment call each turn.

2 Adversarial Verification, Not Optional

An agent that reports its own findings with no one else questioning them has a structural conflict of interest: it was trained to complete the task, not to doubt itself.

In one run against a production target, the attack agent reported 7 critical or high severity findings in a

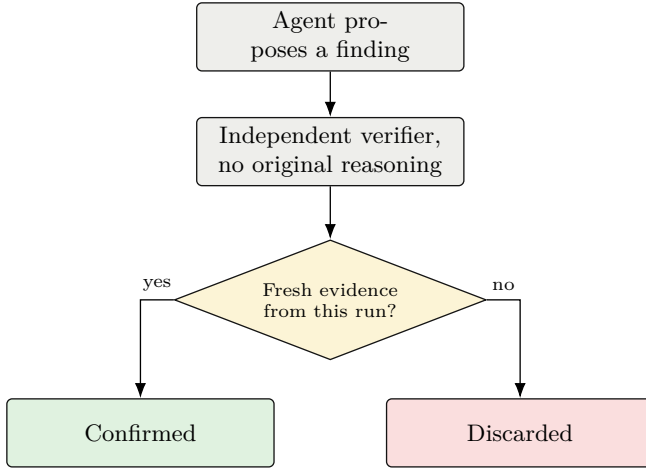


Figure 2: Verification pipeline. The verification step runs on a separate model instance, without the reasoning thread that produced the original finding.

single pass. The pipeline in Figure 2, run immediately after, discarded 6 of the 7 for lack of fresh evidence. The cause, found by reading the agent’s exact prompt: an instruction told it “don’t re-test what’s already confirmed from previous runs.” The agent cited historical findings as if they belonged to this run, without executing a single new tool against them.

The instruction was not wrong by carelessness, it was wrong because it optimized for the wrong thing: saving time by skipping re-confirmation of the obvious, without accounting for the fact that a language model does not distinguish between “this is already known” and “I can claim this as my own.” The fix was not to ban historical context, but to change its function: from fact to hint.

The system, in this case, worked exactly as it should. The verification layer caught 86% of the noise. But a system that needs that layer to avoid lying by accident deserves the same structural distrust applied to any unverified source, even when it arrives dressed as a technical report.

3 An Evidence Hierarchy

Not every “confirmed finding” a security AI agent produces carries the same evidentiary weight. Ranking confirmation techniques by how hard they are to false-positive changes entirely how an AI-generated security report should be read.

When a system has an out-of-band network callback technique available (Figure 3), the correct design decision is to run it every time the scan mode allows, not to leave it as one option among dozens the agent may

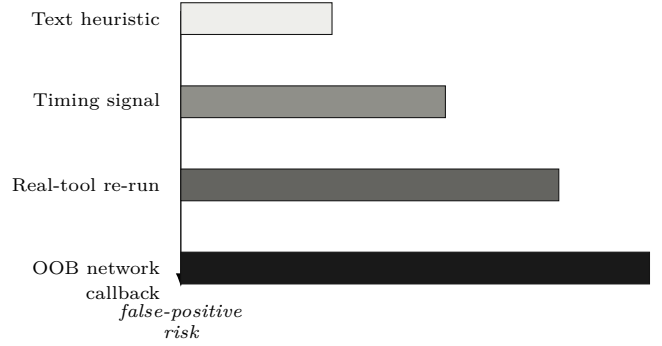


Figure 3: Confidence hierarchy for automated confirmation techniques. The out-of-band network callback, where the target itself makes an outbound request to a unique domain generated for that specific probe, is self-contained proof: it requires no interpretation from the model at all.

or may not choose to call.

4 The Model Isn’t the Bottleneck

The natural assumption is that a bigger, more specialized, or “uncensored” model produces better offensive security results. Put to a controlled test, that assumption did not hold.

The lesson is not that specialized models are bad. It is that reliably orchestrating tools within a structured contract is a distinct skill from “knowing offensive security.” Four local models were benchmarked under the exact same tool-calling contract used by the orchestrator; results are summarized in Table 1.

Neither offensive-specialized model made it as far as measuring recall: one invented its own output syntax instead of using the structured function calls the system expected; the other rejected the request at the architecture level, since its model family does not implement tool-calling at all. Both proved useful as free-text security Q&A or synthesis models, but neither could participate in the agentic loop.

5 Environment Fragility as a Silent Failure

Of all the possible failure modes, the most dangerous is not the model hallucinating. It is a real, correctly-built tool silently failing to run for a reason entirely unrelated to the system’s logic, with nobody noticing because the system keeps producing a report that

Model	Params	Tool-calling contract	Recall / usability	Runtime / verdict
Base	9B	Honored without exception	6/6 known vulnerabilities	21.5 min — reference
“Uncensored” fine-tune	9B	Honored without exception	6/6 known vulnerabilities	60.4 min (2.8×) + 1 new FP from mistaking a broken tool install for a real finding
Offensive-specialized A	7B	Ollama reports it as tool-capable, but it fabricates its own XML instead of filling <code>tool_calls</code>	Unusable as an agent	Complex prompt: 33+ min, never closed the task
Offensive-specialized B	8B	Rejected outright (completion-only architecture, no tool support)	Unusable as an agent	4.9s in pure Q&A mode — correctly cited the CVE queried, but never executes a tool

Table 1: Empirical comparison of four local models under the orchestrator’s exact tool-calling contract. The two “offensive-specialized” models fail before recall can even be measured: one invents its own output format, the other doesn’t declare tool support at all. Both are useful as security Q&A or synthesis models; neither works as an agent.

looks perfectly normal.

A deterministic injection engine failed to run for an entire testing session against a real target. The cause was not in its code: the orchestrating process had been started with an environment variable different from the reference environment, and that single difference made the operating system resolve the binary’s name to the wrong executable sharing that name. The visible symptom was one generic error line buried among dozens of log entries, easy to mistake for “the target wasn’t vulnerable” instead of “the tool never ran.”

This kind of failure drops real coverage below reported coverage, consistently, for as long as the environmental condition persists. The correct response is not to document “remember to export the right variable”; that offloads the responsibility onto human memory. The correct response is for the system itself to guarantee its own environmental conditions when executing each tool, regardless of how it was launched.

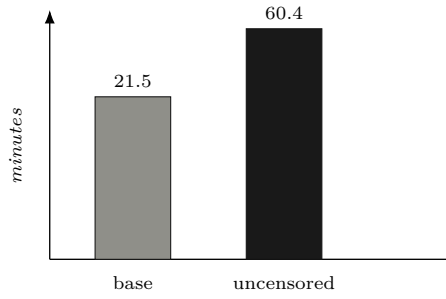
6 Design Principles

1. **Critical capabilities are never optional for the model.** If something must always run, make it a deterministic pipeline step, not just another tool among many.
2. **No finding is trusted in the same pass that**

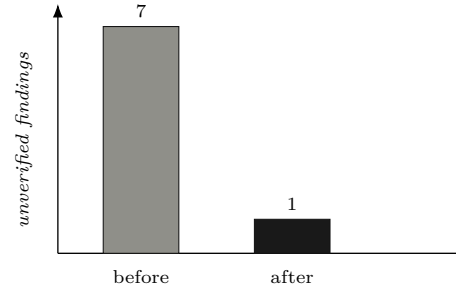
produced it. An independent verification step, without the original reasoning in view, is the difference between a report and a hallucination wearing a report’s clothes.

3. **Rank your evidence, and say so in the report.** A network-callback detection and a text-heuristic detection should not reach the reader under the same “confirmed” label.
4. **Historical context is a hint, never a fact.** If a prompt gives the model permission to skip verifying something, that something will eventually get reported without evidence.
5. **The runtime environment is part of the reliability surface.** A system that depends on its environment being configured “the way it should be” has a fragile dependency disguised as a technical assumption.

Methodological note: the observations above come from operating a single system in production across several work sessions, not from a controlled study across multiple systems. The figures cited are real measurements from those runs; target, client, and tool names are deliberately omitted for confidentiality.



(a) Same target, same recall



(b) Before / after the prompt fix

Figure 4: (a) Two variants of a local ~ 9 B-parameter model against the same lab target: identical recall, but the “uncensored” variant took $2.8\times$ longer and produced one additional false positive. (b) In the same system, fixing a single prompt instruction reduced from 7 to 1 the critical/high findings reaching verification without fresh evidence.